

Clase 6: la clase de los conceptos varios

Métodos y atributos de clase (vs. métodos y atributos de instancia)

Interfaces

Traits

Clases finales

Clases abstractas

Instancia

- También conocido como el “**objeto**”.
- Cada objeto le da un valor propio a los atributos y obtiene un resultado específico para sus métodos.
- Decimos que estos atributos y métodos son “**de instancia**” o que pertenecen a la misma.

```
$persona = new Persona("Alan", "Gonzalez",  
40123456, "3755-123456");
```

```
$persona->getNombre(); // "Alan"
```

```
$persona->getDni(); // 40123456
```

Métodos y atributos de clase

- Los atributos y métodos de clase son elementos que pertenecen a la clase, no a cada objeto.
- Su definición y valores solo pueden realizarse mediante la clase.
- Se definen con la sentencia `static` (también se llaman atributos/métodos estáticos)
- Ejemplo: contador de cuentas.

\$this vs self

\$this	self
Representa una instancia de la clase (objeto)	Representa una clase
Siempre empieza con \$	No utiliza el signo \$
Utiliza el operador de objeto ->	Utiliza el operador estático ::
Luego del operador, los atributos no usan \$	Luego del operador, los atributos estáticos deben usar \$

¿Cuándo usar métodos de clase?

1. Para la creación de clases de utilidad: validaciones, logging, configuración, factories, etc.
2. Para métodos que realicen alguna tarea independiente de los atributos de instancia.
3. Para aquellas operaciones que no dependan de la creación de objetos en una clase.
4. Si necesitamos código que debe ser compartido con todas las instancias y sigue las reglas anteriores.

El problema de la herencia

- En PHP solo existe herencia simple.
- Produce ***acoplamiento fuerte*** entre clases.
 - Difícil de mantener
 - Difícil de extender con nuevas funcionalidades
 - Difícil de corregir errores surgidos mediante el mecanismo de herencia (en algunos escenarios)
- Para solventar este caso podemos utilizar **Traits** o **Interfaces**

Interfaces vs Traits

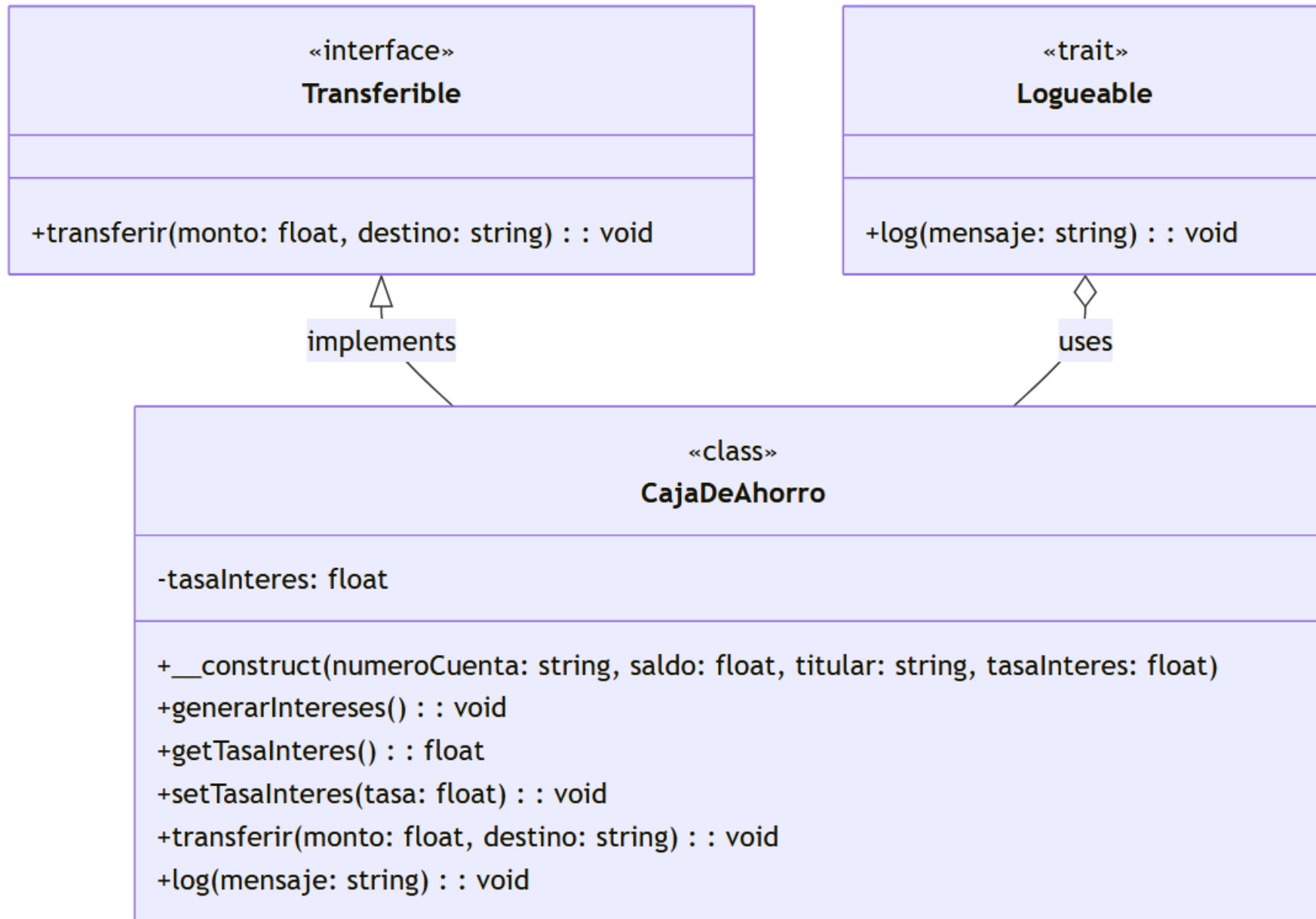
Concepto	Interface	Trait
Definición	Contrato: define los métodos que debe implementar la clase	Contenedor: define código que las clases pueden reutilizar
Implementación en la clase	Las clases deben definir todos los métodos de la interface	La clase no define nada, se usa el código ya definido
Sentencia	Implements	Use
Combinación	Una clase puede implementar múltiples interfaces	Una clase puede usar múltiples traits
Herencia de atributos	No, solo se trabajan con métodos	Si, se pueden heredar atributos y métodos

Uso de un trait

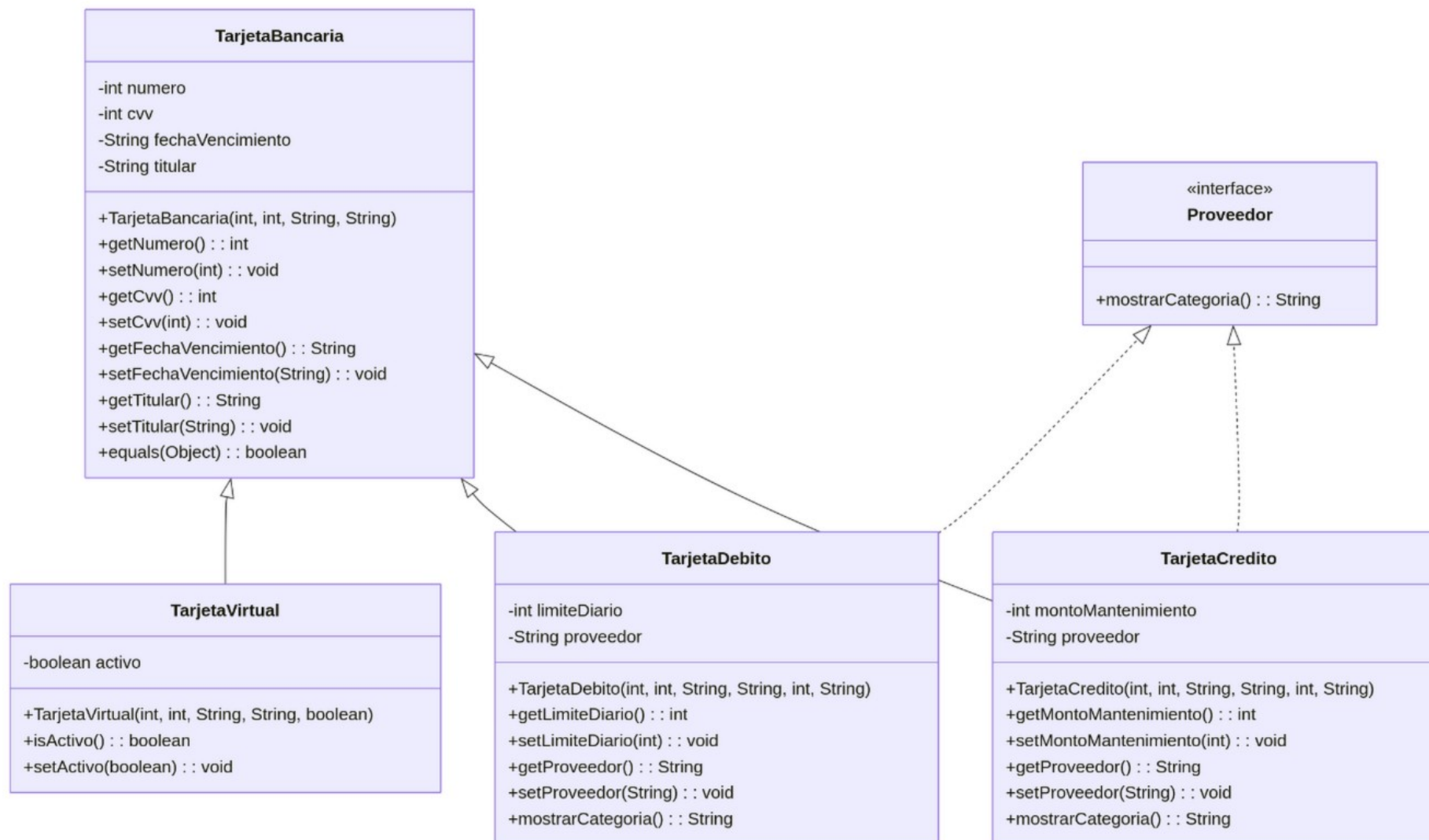
```
trait NombreTrait {  
    public function funcion() { // cuerpo }  
}  
// Usarlo en una clase  
class NombreClase {  
    use NombreTrait;  
    // En cualquier método $this->funcion()  
}
```


Uso de una interface

```
interface Animal {  
    public function sonido(): void;  
}  
  
class Gato implements Animal {  
    public function sonido(): void {  
        echo "Miau";  
    }  
}  
  
$gato = new Gato();  
$gato->sonido(); // Miau
```



Ejercicio 2



Como implementar mostrarCategoria()

- Dependiendo el proveedor, se deberá mostrar una u otra categoría:
- Si el proveedor es VISA, la categoría es Premium
- Si el proveedor es Mastercard, la categoría es Platino
- Si es otro proveedor, la categoría es Común

Clases finales

- Una **clase final** es aquella que no puede ser extendida mediante herencia por otra clase.
- También se puede agregar **final** a un método para impedir que al ser heredado sea sobrescrito por la clase hija.

```
final class ClaseA {  
    // Cuerpo de la clase A  
}  
class ClaseB extends ClaseA {} // Error fatal
```

¿Para qué se usan?

1. Prevenir una interminable serie de herencias entre clases (mala práctica).
2. Favorecer la composición y el uso de interfaces/traits.
3. Proteger el encapsulamiento de los métodos originales.
4. Para favorecer un diseño robusto de los métodos y no delegar todo en una “futura y posible” herencia.

[When to declare classes final \(ocramius.github.io\)](https://ocramius.github.io)

Clases abstractas

- Una **clase abstracta** es aquella de la cual no se pueden crear instancias/objetos.
- Puede contener atributos y métodos de instancia como cualquier clase.
- Se puede usar **abstract** en métodos, los cuales solo podrán definir su firma, pero no su lógica.
- Las clases hijas que hereden su estructura de una clase abstracta se verán obligadas a implementar todos los métodos abstractos de la clase padre.

Ejemplo de clase abstracta

```
abstract class Vehiculo {  
    protected $combustible;  
    public function setCombustible($combustible) {  
        $this->combustible = $combustible;  
    }  
    abstract public function encenderMotor();  
}  
class Auto extends Vehiculo {  
    public function encenderMotor() {  
        echo 'Motor encendido con ' . $this->combustible;  
    }  
}
```


Uso de la clase Auto

```
$auto = new Auto();  
$auto->setCombustible('gasoil');  
$auto->encenderMotor();  
// Motor encendido con gasoil
```

Ejercicio final de la Unidad